Theme 11 — Verdict→Action Solution Document

Verdict→Action — From Court Judgments to Verified Action Plans

PanIIT AI for Bharat Hackathon — Theme 11 Submission

---

1. Executive Summary

Court judgments are among the most consequential legal artifacts in Indian governance — a single order from the Supreme Court or a High Court can mandate dozens of obligations across state departments, each with its own deadline, responsible party, and compliance requirement. Yet the gap between *judgment pronounced* and *obligation executed* is routinely measured in months or years, not because departments don't want to comply, but because extracting, tracking, and verifying each obligation against its verbatim source is a manual, error-prone process spread across PDFs, spreadsheets, and departmental memory.

Verdict→Action is an AI platform that closes this loop. Upload a judgment PDF, and the system extracts every legal obligation — technical, financial, compliance — with its source excerpt, its parsed deadline, its responsible party, and a confidence-scored reasoning. Officers then verify, track status, and log updates against each obligation. Every decision is traceable back to the exact paragraph of the judgment, creating an audit-ready workflow that scales from one judgment to a hundred.

Our differentiators are three: (1) **verbatim source traceability** — every obligation displays the exact paragraph from the judgment, with page reference, so a reviewer can verify the AI didn't hallucinate; (2) **deadline parsing** — phrases like "within 90 days of pronouncement" or "before the monsoon session" are converted to actual dates using pattern rules grounded in the judgment date; and (3) **verification workflow** — no obligation is ever "active" until a human officer explicitly marks it verified, ensuring the AI augments rather than replaces legal judgment.

2. Problem Deep Dive

The pain point

When a court delivers a judgment, the directives spread across dozens of paragraphs are the *actionable* output. A single 150-page environmental pollution judgment can contain 40+ obligations: publish revised SOPs within 60 days, file quarterly compliance reports, increase inspection frequency, appoint a monitoring officer, compensate affected parties. Each obligation has a **type** (deadline-bound vs continuous), a **deadline** (verbatim text vs parseable date), a **responsible party** (ministry, board, department), a **priority** (critical vs medium), and **verbatim source text** that justifies the obligation exists.

Today, the process is: 1. Law officer reads the judgment (days of work) 2. Creates an action-plan Word document (prone to omission) 3. Circulates via email to responsible departments 4. Departments acknowledge verbally or by email; no central tracking 5. Courts occasionally ask compliance status; answers take weeks to assemble

This manual pipeline fails in three predictable ways: - **Missed obligations:** one paragraph buried in a 150-page judgment gets overlooked; compliance later challenged - **Wrong deadlines:** "within 90 days" sometimes computed from pronouncement, sometimes from gazette notification, sometimes forgotten entirely - **Broken traceability:** when auditors ask "where does this obligation come from?", officers scramble through PDFs

Stakeholders

Direct users: Law officers, compliance officers, secretaries of ministries and departments responsible for implementing court directives. **Indirect beneficiaries:** Citizens affected by judgments (pollution victims, labour rights claimants, procurement whistleblowers); courts themselves when checking compliance status.

Regulatory and institutional context

- **Supreme Court and High Court judgments:** the primary source of obligations; the Supreme Court alone issues hundreds of judgments with directives every year.
- **Tribunal judgments (NGT, CAT, CCI, TDSAT):** sector-specific; equally binding; often more frequent.
- **RTI Act 2005:** citizens have the right to know compliance status; Verdict→Action makes this answerable in seconds.
- **Indian Evidence Act + audit trail requirements:** any system supporting legal compliance must produce reconstructible decision histories.

3. Solution Architecture

Pipeline stages

Stage 1 — Ingestion. A law officer uploads a PDF. The system extracts text via `pdf-parse` (JavaScript-native, no OCR sidecar needed for typed judgments). Page count and full text are persisted. Status transitions UPLOADED → PARSING → PARSED.

Stage 2 — Extraction. `lib/ai.ts` (mock-first with keyword rules; LLM-compatible interface) extracts: - **Parties** — petitioners, respondents, intervening parties, the State, regulatory bodies - **Obligations** — per directive: type, priority, title, description, deadline text, source excerpt (verbatim, ≤400 chars), source page, responsible party, reasoning, confidence

Stage 3 — Deadline Parsing. `lib/deadlines.ts` converts deadline phrases into actual dates: - "within N days/weeks/months" → judgment_date + N units - "by DD/MM/YYYY" → parsed directly - "before the monsoon session" → flagged for manual clarification (no silent interpretation)

Stage 4 — Storage. Prisma persists Judgment → Parties → Obligations with relational integrity. `ObligationUpdate` captures status changes over time, building the audit trail.

Stage 5 — Verification. No obligation is treated as active until an officer explicitly reviews the source excerpt, confirms the AI's reading, and marks `verified = true`.

Stage 6 — Tracking. Officers update obligation status as work progresses (PENDING → IN_PROGRESS → COMPLETED), with optional evidence URLs. The dashboard shows real-time compliance state.

Data model

- **Judgment** — uploaded PDF, extracted text, metadata, status, obligations
- **Party** — named party with role (petitioner/respondent/State/intervener)
- **Obligation** — the core entity: type, priority, status, deadline (verbatim + parsed date), responsible party ID, **source excerpt + source page**, reasoning, confidence, verified flag, verifier notes
- **ObligationUpdate** — every status change creates an entry with note, evidence URL, updated-by, timestamp

Why verbatim traceability matters

An obligation's `sourceExcerpt` is the exact paragraph from the judgment (up to 400 characters). The UI displays it prominently in every obligation detail. This means: - The AI cannot silently hallucinate an obligation - A reviewer can verify each extraction in seconds - When auditors ask "where did this obligation come from?", the answer is a verbatim quote with a page number

This is the single most important design decision in the system.

Technology choices

Choice	Justification
Next.js 15 App Router	Single repo, deploys to Vercel free tier, server-side Prisma queries.
Prisma + SQLite	Zero-infra prototype; PostgreSQL-portable.
`pdf-parse`	JavaScript-native PDF text extraction. No Python sidecar. Handles typed judgments (99% of Supreme Court/HC output).
`pdfkit`	For generating test PDFs from sample judgment text files — shipping-ready sample data.
Tailwind v3 + shadcn/ui + Tremor	Production-quality UI. Tremor Cards and BarLists for the obligation tracker.
Mock AI first	Deterministic, reproducible, zero-cost demos. Real LLM drops in via `USE_MOCK_AI=false`.
Deadline parser as pure function	Testable, auditable, no black-box behaviour.

Handling scanned judgments (production path)

The MVP handles typed PDFs (Supreme Court, High Court, NGT — all produce typed PDFs). For scanned judgments (district courts, older archives), the production integration plugs Docling + PaddleOCR in before `pdf-parse`. This path is commented in `lib/pdf.ts` and documented here.

4. Government Feasibility & Deployment

Alignment with Indian judicial and administrative infrastructure

- **Integration with eCourts Services:** Supreme Court and High Court judgments are published on eCourts with permanent URLs. Our ingestion can accept a URL and download the PDF directly — no manual upload needed for 95% of judgments.
- **Ministry-specific deployment:** each ministry can host its own Verdict→Action instance, importing only judgments relevant to its mandate. No centralisation required.
- **PM GatiShakti / DBT dashboards:** obligation compliance data can be exposed via API to government transparency dashboards.
- **Data sovereignty:** runs on MeghRaj. No external API calls when `USE_MOCK_AI=true`; even real LLM usage can route through on-premise instances for sensitive judgments.

90-day pilot

Days 1–30 — Deployment. Pilot with one ministry (e.g., Environment & Forests — NGT judgments are frequent and high-stakes). Import the last 12 months of relevant judgments. Train 3–5 law officers on the verification workflow.

Days 31–60 — Calibration. Compare AI-extracted obligations against the ministry's existing compliance tracking. Measure precision (AI obligation matches reality), recall (AI catches obligations humans missed), and time-to-verify (minutes per obligation vs hours manually).

Days 61–90 — Scale. Expand to 3 more ministries. Enable automatic ingestion from eCourts. Hand off to the ministry's internal law cell.

Cost

Infrastructure: ~₹8,000/month for a VM + managed Postgres. LLM costs at scale: for a ministry processing 200 judgments/year at ~₹2/judgment with Claude Haiku, total LLM cost is negligible (~₹400/year). Total pilot year-1 cost per ministry: **~₹1.2 lakh** including deployment.

Change management

The UI is designed for a single-persona workflow: a law officer who already reads judgments. The verification workflow maps onto their existing mental model (read → identify obligations → track compliance). Training time: one hour.

5. Prototype Description

What the prototype demonstrates

- PDF judgment upload with `pdf-parse` text extraction
- Pre-seeded demo with 3 realistic judgments (environmental pollution, labour rights, procurement transparency), 8 parties, 31 obligations
- Dashboard with Tremor metric cards + priority donut + upcoming deadlines feed
- Judgment detail with 4 tabs: **Overview** (metadata + counts), **Obligations** (filterable with verbatim source excerpts), **Source Document** (full judgment text with obligation anchors), **Parties** (role-grouped with assigned obligations)
- Cross-judgment obligation tracker grouped by status, priority, or deadline
- Verification workflow (toggle verified flag with notes)
- Status change history (ObligationUpdate timeline)

API surface

```
POST   /api/judgments/upload          PDF upload + extraction + parse
GET    /api/judgments                 {total, judgments}
GET    /api/judgments/[id]            full judgment with obligations + pa
DELETE /api/judgments/[id]
GET    /api/obligations               cross-judgment filter {total, obliga
GET    /api/obligations/[id]          full detail with source + reasoning
PUT    /api/obligations/[id]/verify   mark verified with notes
PUT    /api/obligations/[id]/status   status change + note + evidence URL
GET    /api/dashboard/stats           metrics for dashboard
GET    /api/dashboard/deadlines       upcoming deadline feed
```

Sample data

3 realistic Indian court judgment text files: - **env-pollution-judgment.txt** — 15 pages, 8 obligations (pollution board reporting, cleanup deadlines, compensation) - **labor-rights-judgment.txt** — 10 pages, 6 obligations (minimum wage implementation, inspection cadence) - **procurement-judgment.txt** — 12 pages, 7 obligations (tender transparency, e-procurement rollout)

Plus a `data/judgments-to-pdf.ts` utility that uses `pdfkit` to generate real PDFs — so the upload flow works end-to-end for judges.

6. Scalability & Long-Term Impact

Volume

- Per ministry: typically 50–500 judgments/year → trivial load
- National scale across 50+ ministries/agencies: ~25,000 judgments/year → single managed Postgres instance handles comfortably
- Multi-tenancy: namespace data by ministry_id in the Prisma schema (one-line change)

Multi-language

India's courts increasingly issue judgments in regional languages. The architecture swaps in multilingual models in `lib/ai.ts` without upstream changes. The deadline parser is language-aware (Hindi and Marathi equivalents of "within N days" are one regex each).

Long-term impact

- **Compliance acceleration:** obligations tracked in days, not months
- **Citizen accountability:** RTI responses on judgment compliance become instant
- **Court effectiveness:** judges can issue directives with confidence they'll be tracked
- **Reduced contempt-of-court risk:** missed obligations surface before they become contempt

7. Innovation Highlights

1. **Verbatim source excerpt per obligation** — the unique design choice that makes the system trustworthy
2. **Deadline parser with "needs clarification" fallback** — no silent misinterpretation of ambiguous deadlines
3. **Verification flag before activation** — AI augments, never replaces, legal judgment
4. **Cross-judgment obligation tracker** — officers see the whole compliance landscape, not just per-judgment views
5. **ObligationUpdate timeline** — every status change is an immutable record with note + evidence
6. **Mock-AI-first design** — deterministic, reproducible, zero-cost demos

8. Risks and Mitigations

Risk	Mitigation
Hallucinated obligations (AI extracts something not in the judgment)	Verbatim source excerpt displayed alongside every obligation; officer must verify before it's active.
Wrong deadline parsing	Deadline parser has explicit "ambiguous — human review" fallback. Judgment date is used as anchor, never inferred.
Scanned / poorly-OCR'd judgments	MVP handles typed PDFs (95% of HC/SC output). Production path: Docling + PaddleOCR pre-stage. Documented in `lib/pdf.ts`.
Judgments in regional languages	Multilingual model swap in `lib/ai.ts`. Deadline parser extensible with per-language regex packs.
Responsibility confusion (who owns this obligation?)	Party matching against extracted parties; unmatched obligations flagged for officer assignment.
Compliance updates mismatched to reality	`ObligationUpdate` carries evidence URL and updated-by; audit-grade immutable log.
Integration with legacy departmental systems	Open REST API; JSON export; CSV export; no vendor lock-in.

9. Team & References

Team: Full-stack engineering with interest in legal tech and public administration. Open to partnering with a ministry law cell for the 90-day pilot.

References: - Indian Evidence Act 1872 (audit trail requirements) - RTI Act 2005 (citizen access to compliance status) - eCourts Services (judgment repository — https://ecourts.gov.in) - Supreme Court of India Rules 2013 (judgment structure) - National Green Tribunal Act 2010 (for NGT-specific obligations) - `pdf-parse` library (MIT-licensed, JavaScript PDF text extraction)

Repo: https://github.com/sridhar7601/verdict-to-action **Stack:** Next.js 15 · TypeScript · Prisma · SQLite · Tailwind v3 · shadcn/ui · Tremor · `pdf-parse` · `pdfkit`